

Лабораторная работа № 2, по курсу дискретного анализа: Сбалансированные деревья

Выполнил студент группы М8О-209Б-23 МАИ *Борисов Денис Сергеевич*.

Условие

Необходимо создать программную библиотеку, реализующую указанную структуру данных, на основе которой разработать программу-словарь. В словаре каждому ключу, представляющему из себя регистронезависимую последовательность букв английского алфавита длиной не более 256 символов, поставлен в соответствие некоторый номер, от 0 до $2^{64} - 1$. Разным словам может быть поставлен в соответствие один и тот же номер.

Программа должна обрабатывать строки входного файла до его окончания. Каждая строка может иметь следующий формат:

+ **word 34** — добавить слово «word» с номером 34 в словарь. Программа должна вывести строку «OK», если операция прошла успешно, «Exist», если слово уже находится в словаре.

- **word** — удалить слово «word» из словаря. Программа должна вывести «OK», если слово существовало и было удалено, «NoSuchWord», если слово в словаре не было найдено.

word — найти в словаре слово «word». Программа должна вывести «OK: 34», если слово было найдено; число, которое следует за «OK:» — номер, присвоенный слову при добавлении. В случае, если слово в словаре не было обнаружено, нужно вывести строку «NoSuchWord».

! **Save /path/to/file** — сохранить словарь в бинарном компактном представлении на диск в файл, указанный параметром команды. В случае успеха, программа должна вывести «OK», в случае неудачи выполнения операции, программа должна вывести описание ошибки (см. ниже).

! **Load /path/to/file** — загрузить словарь из файла. Предполагается, что файл был ранее подготовлен при помощи команды Save. В случае успеха, программа должна вывести строку «OK», а загруженный словарь должен заменить текущий (с которым происходит работа); в случае неуспеха, должна быть выведена диагностика, а рабочий словарь должен остаться без изменений. Кроме системных ошибок, программа должна корректно обрабатывать случаи несовпадения формата указанного файла и представления данных словаря во внешнем файле.

Для всех операций, в случае возникновения системной ошибки (нехватка памяти, отсутствие прав записи и т.п.), программа должна вывести строку, начинающуюся с «ERROR:» и описывающую на английском языке возникшую ошибку.

Различия вариантов заключаются только в используемых структурах данных: AVL-дерево.

Вариант: 1-1

Метод решения

Рассмотрим структуру данных AVL дерево. AVL дерево - это бинарное дерево поиска, в котором модуль разности между высотой правого поддерева и высотой левого не превышает единицу. Мой метод решения предполагает под собой хранение ключа и значения, полученных из ввода, а также высоты поддерева в узле. Соответственно, требуется реализовать операции вставки, удаления, балансировки и поиска в AVL дереве, а также сериализации и десериализации дерева для чтения и записи в файл. Балансировка в решении осуществляется с помощью баланс фактора. Баланс фактор - это разница между высотами правого и левого поддерева. Также для балансировки применяются процедуры левого и правого поворота.

Поиск в AVL дереве аналогичен поиску в бинарном дереве поиска. Операция вставки и удаления аналогичны операциям вставки в бинарном дереве поиска, однако после них при необходимости нужно выполнить балансировку.

Реализованное AVL дерево используется в качестве словаря, над которым производятся операции добавления, удаления, поиска, записи в файл или чтения из файла в зависимости от команд пользователя. Словарь хранит пары ключ-значение. Ключом выступает строка (не более 256 значащих символов), значением беззнаковое 8-ми байтовое целое.

Описание программы

Данная программа представляет собой реализацию словаря на основе сбалансированного бинарного дерева поиска типа AVL. Программа позволяет добавлять слова с их числовыми значениями, удалять слова, искать значения по ключу, а также сохранять и загружать словарь из файла.

Структура программы

Основу программы составляет шаблонный класс `AVL<K, T>`, реализующий сбалансированное бинарное дерево AVL, где:

- `K` — тип ключа (в данной реализации используется `std::string`)
- `T` — тип значения (в данной реализации используется `unsigned long long`)

Класс AVL содержит следующие основные методы:

- `insert(const K &k, const T &v)` — добавление пары ключ-значение
- `remove(const K &k)` — удаление элемента по ключу
- `find(const K &k)` — поиск значения по ключу
- `serialize(std::ofstream &ofs)` — сохранение дерева в бинарный файл
- `deserialize(std::ifstream &is)` — загрузка дерева из бинарного файла

Алгоритмическая основа

AVL дерево — это сбалансированное бинарное дерево поиска, в котором для каждого узла разница высот левого и правого поддеревьев (фактор баланса) не превышает 1 по модулю. Для поддержания баланса в программе реализованы следующие операции:

- Правое вращение (`rightRotate`)
- Левое вращение (`leftRotate`)
- Балансировка (`balance`) — применение вращений для восстановления баланса

Функциональность программы

Программа поддерживает следующие команды:

- `+ <ключ> <значение>` — добавление слова и его значения
- `- <ключ>` — удаление слова
- `<ключ>` — поиск значения по ключу

- ! Save <имя_файла> — сохранение словаря в файл
- ! Load <имя_файла> — загрузка словаря из файла

Все ключи приводятся к нижнему регистру перед обработкой, что делает поиск нечувствительным к регистру.

Дневник отладки

В ходе выполнения работы было затрачено много времени на осмысление и реализацию алгоритма, однако получилось потратить достаточно мало попыток на решение задачи. В ходе сдачи работы я столкнулся с ошибкой компиляции, она была вызвана недостаточной оптимизацией ввода данных. Проблему удалось быстро заметить и устранить благодаря опыту ранее сданных задач.

Тест производительности

Оценка сложности алгоритма

Для оценки сложности операций в AVL-дереве нам потребуется установить связь между высотой дерева и количеством узлов. Для этого мы воспользуемся числами Фибоначчи и свойствами золотого сечения.

Определим последовательность чисел Фибоначчи F_n следующим образом: $F_0 = 0$
 $F_1 = 1$
 $F_n = F_{n-1} + F_{n-2}, n \geq 2$

Известно, что отношение соседних чисел Фибоначчи при $n \rightarrow \infty$ стремится к золотому сечению: $\lim_{n \rightarrow \infty} \frac{F_{n+1}}{F_n} = \varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$

Обозначим через $N(h)$ минимальное количество узлов в AVL-дереве высоты h . Докажем по индукции, что $N(h) \geq F_{h+2} - 1$. Для $h = 0$ имеем одиночный узел, т.е., $N(0) = 1$. При этом $F_2 - 1 = 1 - 1 = 0$, условие $N(0) \geq F_2 - 1$ выполняется. Для $h = 1$ имеем как минимум 2 узла, т.е., $N(1) = 2$. При этом $F_3 - 1 = 2 - 1 = 1$, условие $N(1) \geq F_3 - 1$ выполняется. Допустим, что $N(k) \geq F_{k+2} - 1$ для всех $k < h$. Минимальное количество узлов в AVL-дереве высоты h складывается из корня, левого поддерева высоты $h - 1$ и правого поддерева высоты $h - 2$ (минимально возможная высота при условии, что разница высот не превышает $N(h) = F_{h+2} - 1$).

Теперь мы можем оценить максимальную высоту h AVL-дерева с n узлами:

$$n \geq N(h) \geq F_{h+2} - 1$$

Из теории чисел Фибоначчи известно, что $F_n \approx \frac{\varphi^n}{\sqrt{5}}$ для больших n , где $\varphi = \frac{1+\sqrt{5}}{2}$ — золотое сечение.

Подставляя это приближение, получаем: $n + 1 \geq F_{h+2}$

$$n + 1 \geq \frac{\varphi^{h+2}}{\sqrt{5}}$$

$$(n + 1)\sqrt{5} \geq \varphi^{h+2}$$

$$\log_{\varphi}((n + 1)\sqrt{5}) \geq h + 2$$

Отсюда:

$$h \leq \log_{\varphi}((n + 1)\sqrt{5}) - 2 = \log_{\varphi}(n + 1) + \log_{\varphi}(\sqrt{5}) - 2 \approx \log_{\varphi}(n) - 1.1$$

Следовательно, $h \in O(\log n)$.

Оценка сложности операций

Поиск в AVL-дереве Алгоритм поиска в AVL-дереве аналогичен поиску в обычном бинарном дереве поиска: мы начинаем с корня и на каждом шаге сравниваем ключ с текущим узлом, двигаясь либо влево, либо вправо.

Так как глубина каждой ветви дерева не превышает $h \in O(\log n)$, то временная сложность операции поиска составляет:

$$T(n) \in O(\log n)$$

Вставка в AVL-дерево Операция вставки состоит из двух фаз:

1. Стандартная вставка в бинарное дерево поиска — $O(\log n)$.

2. Балансировка дерева, включающая пересчет высот и выполнение вращений.

Пересчет высот и проверка баланса выполняются за $O(1)$ для каждого узла на пути от добавленного узла к корню. Вращения также выполняются за $O(1)$.

Путь от нового узла до корня не превышает высоту дерева, которая $O(\log n)$. В худшем случае нам нужно выполнить $O(1)$ вращений на каждом уровне при подъеме от листа к корню.

Таким образом, общая временная сложность вставки:

$$T(n) \in O(\log n)$$

Удаление из AVL-дерева Операция удаления также состоит из двух фаз:

1. Стандартное удаление из бинарного дерева поиска — $O(\log n)$.
2. Балансировка дерева от точки удаления до корня.

Как и при вставке, балансировка требует $O(\log n)$ операций проверки баланса и вращений в худшем случае.

Следовательно, временная сложность удаления:

$$T(n) \in O(\log n)$$

Оценка производительности

Сравним созданное в результате выполнения задания AVL дерево со стандартным контейнером `std::map`. Это имеет смысл так как в его основе лежит красно-черное дерево, которое тоже является сбалансированным. Замеры производительности проводились на операциях: вставки, поиска, удаления 4-х байтовых целочисленных значений.

Из графиков видно, что, как и ожидалось, AVL дерево проигрывает красно-черному в операциях вставки, удаления, однако выигрывает на операции поиска.

Это происходит в силу различий в алгоритмах балансировки: в красно-черных деревьях она происходит гораздо реже, чем в AVL. Как следствие, вставка и удаление происходит быстрее. Но при этом и высота красно-черного дерева больше высоты соответствующего AVL дерева. Отсюда вытекает большее время поиска.

Не логарифмическая зависимость на графиках, по моему мнению, получается из-за довольно частых выделений и освобождений памяти на куче, а это не самая дешевая в плане времени операция. К тому же графики имеют очень похожую форму, что говорит об асимптотически одинаковой скорости роста времени выполнения операций.

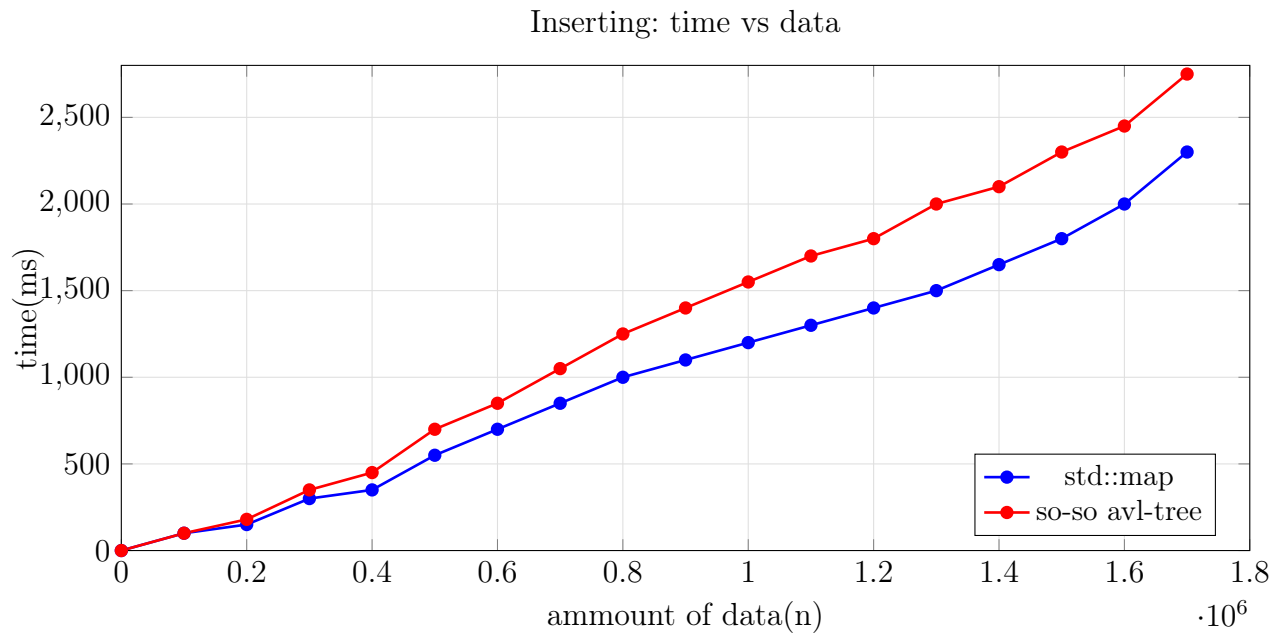


Рис. 1: Сравнение времени вставки элементов в AVL-дерево и std::map

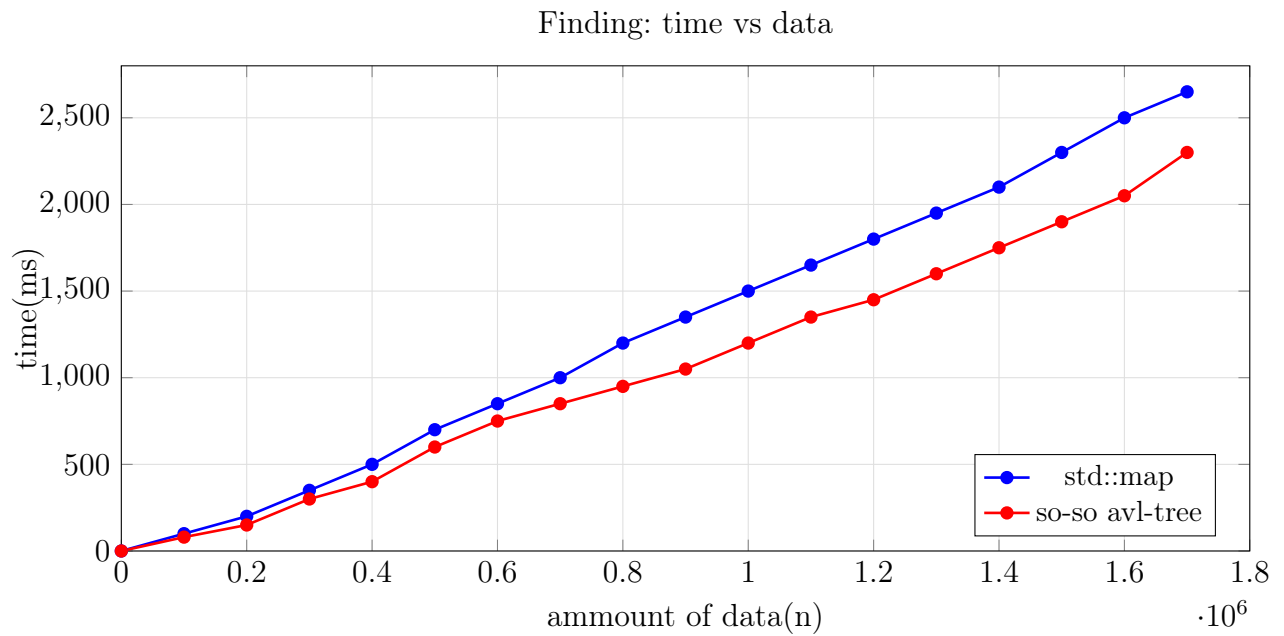


Рис. 2: Сравнение времени поиска элементов в AVL-дерево и std::map

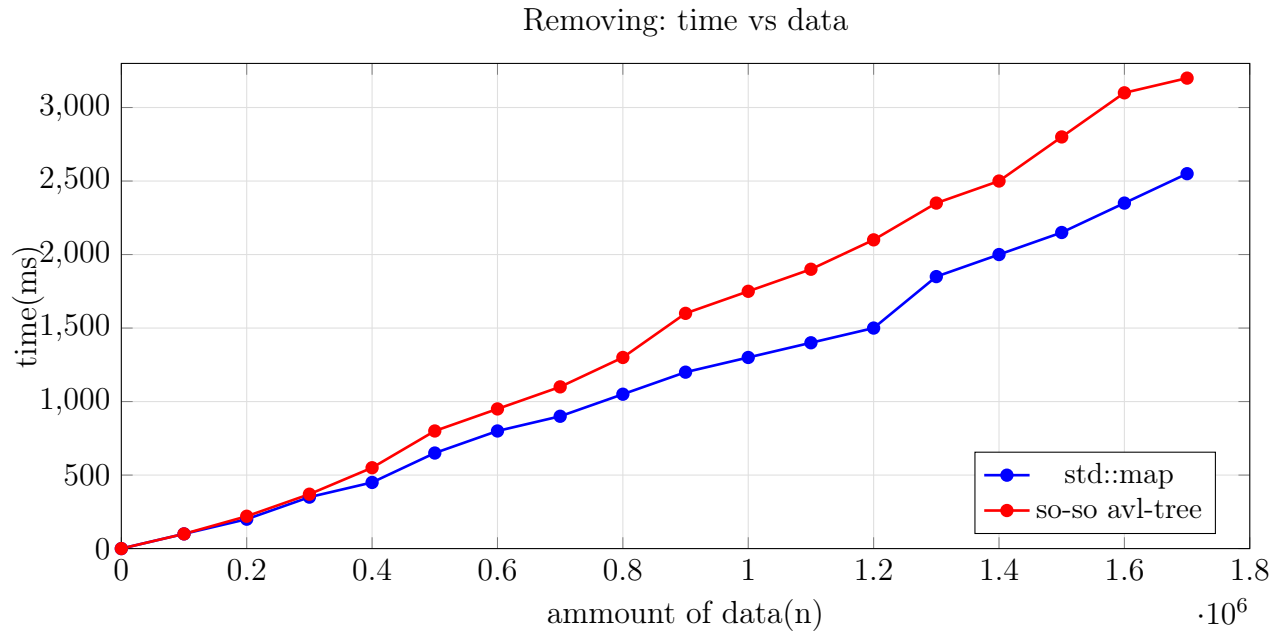


Рис. 3: Сравнение времени удаления элементов из AVL-дерева и `std::map`

Выводы

В результате выполнения лабораторной работы, я изучил реализацию AVL дерева и реализовал на его основе словарь, провел анализ своей реализации и сравнил ее со стандартным контейнером `std::map`.

Все три основные операции в AVL дереве (поиск, вставка и удаление) имеют логарифмическую временную сложность $O(\log n)$. Это гарантируется благодаря свойству сбалансированности AVL деревьев, которое обеспечивает, что высота дерева всегда ограничена логарифмической функцией от числа узлов. Теоретически нижняя граница высоты определяется свойствами чисел Фибоначчи и золотого сечения, что делает AVL дерево одной из наиболее эффективных структур данных для операций поиска, вставки и удаления элементов. Однако сравнительный анализ показал уязвимости AVL дерева по сравнению с другими сбалансированными структурами данных.